PathAble Edge System

Indoor Assistive Navigation Operations & Teammate Manual

OPENCV AI COMP 2026

1. System Architecture & Dual-Loop Concept

PathAble uses a Client-Edge-Cloud architecture tailored for real-time assistive guidance for visually impaired users. Operations are divided into two distinct loops to balance instant obstacle safety with global path planning:

- **Fast Perception Loop (Local Edge)**

  Runs on local Python server (8080). Processes 640×480 mobile camera frames at 5 FPS to deliver immediate guidance:

  - **ArUco Localization:** Fast marker scanning (1.1 ms) at corridor junctions ($A \rightarrow F$).
  - **YOLOv11n-seg ONNX:** Obstacle segmentation (80 ms) with smart frame skipping.

- **Slow Navigation Loop (Graph & IoT)**

  Manages global positioning and environmental awareness via NetworkX graph routing:

  - **Dijkstra Pathfinding:** Recalculates shortest corridor routes on dynamic blockages.
  - **IoT Telemetry:** Merges ESP32 ToF distance and MPU6050 heading data.

2. Three-Terminal System Launch Sequence

To run the production pipeline during testing, open **three separate PowerShell windows** inside your project folder (`PATHABLE_WORK\production`):

Terminal 1: Static Web Server (Port 8000)

```
python -m http.server 8000
```

Serves `index.html` to mobile browsers across the network.

Terminal 2: PathAble Edge Server (Port 8080)

```
python production_server.py
```

Executes ArUco localization, YOLO ONNX perception, and Dijkstra rerouting engine.

Terminal 3: Secure HTTPS Tunnel (ngrok)

```
ngrok http 8000 --url=grandly-macaroni-splinter.ngrok-free.dev
```

Creates an HTTPS bridge to bypass mobile camera security restrictions.

3. Mobile Client Deployment & Initial Connection

Mobile web browsers enforce strict security rules that block camera access over plain `http://` IP connections. Follow this connection checklist when testing on iOS or Android:

> **Smartphone Setup Procedure**
>
> 1. Ensure your testing phone is connected to the **same Wi-Fi network** as the host laptop.
> 2. Open Chrome or Safari on your phone and enter:
>   `https://grandly-macaroni-splinter.ngrok-free.dev/index.html`
> 3. Tap the blue "**Start Navigation & Camera**" button. Accept the browser prompt for camera access.
> 4. Verify the status panel displays: `"Connected to Python Server!"` and rear video feed appears.

4. Corridor Map Layout & ArUco Marker Grid

The system utilizes a 6-node topological graph defined in `map_graph.json`. Print or display the following OpenCV ArUco markers (Dictionary: `DICT_4X4_50`):

Marker ID	Graph Node	Corridor Location	Connecting Edges	Walking Distance
`ID 0`	**Node A**	Main Corridor Entrance	A → B, A → E	5.0 m / 7.0 m
`ID 1`	**Node B**	Mid Corridor Junction 1	B → A, B → C, B → E	5.0 m / 5.0 m / 4.0 m
`ID 2`	**Node C**	Mid Corridor Junction 2	C → B, C → D	5.0 m / 5.0 m
`ID 3`	**Node D**	Target Destination	D → C, D → F	5.0 m / 7.0 m
`ID 4`	**Node E**	Alternate Bypass East	E → A, E → B, E → F	7.0 m / 4.0 m / 6.0 m
`ID 5`	**Node F**	Alternate Bypass South	F → E, F → D	6.0 m / 7.0 m

> **Physical Marker Mounting Guidelines**
>
> Mount markers at chest height ($1.2\text{m} - 1.4\text{m}$) at hall intersections. Ensure a minimum **2cm white border (quiet zone)** around the outer black frame. Do not laminate with glossy foil to avoid specular reflection under fluorescent lighting.

5. Demonstration & Evaluation Test Cases

Test 1: Clear Hallway

Walk down corridor with no obstacles. Verify the app maintains `CONTINUE` status and remains silent until reaching a junction marker.

Test 2: Single Obstacle

Place a chair in center lane. System should segment bounding box and speak: *"Obstacle ahead. Step slightly right."*

Test 3: Full Blockage

Block left, center, and right zones. Engine removes edge in graph, triggers Dijkstra, and speaks: *"Route blocked. Turn towards junction E."*

6. Troubleshooting & Common Issues Matrix

Symptom	Root Cause	Immediate Fix Procedure
Camera prompt does not appear on phone	Opened via plain HTTP IP address or ngrok tunnel disconnected	Use HTTPS URL: `https://grandly-macaroni-splinter.ngrok-free.dev/index.html`. Restart ngrok if tunnel timed out.
`TypeError: handler() missing 'path'`	`websockets` v16+ parameter signature mismatch	Update `production_server.py` handler definition to: `async def handler(websocket, path=None):`
ArUco marker visible but not scanning	Image compressed/blurred or dictionary mismatch	Ensure canvas size in `index.html` is 640×480 with JPEG quality 0.75. Confirm marker belongs to `DICT_4X4_50`.
`Reshape2LayerImpl` OpenCV DNN Crash	ONNX blob size mismatch (fed 320x320 instead of 640x640)	Set `blob_size = 640` inside `process_onnx_inference()` in `production_server.py`.
App voice feedback silent	Mobile browser speech synthesis uninitialized	User must physically tap the "Start Navigation & Camera" button on page load to unlock audio permissions.

7. Latency & Performance Calibration

The Fast Loop execution speed is logged directly to the server terminal as `[PERF]`. Target performance figures during live demonstrations:

ArUco Landmark Scanning

Target Latency: $1.0\text{ ms} - 3.0\text{ ms}$
Executes on 100% of incoming frames to guarantee real-time junction detection without missing turn markers while walking.

YOLO ONNX Obstacle Perception

Target Latency: $75\text{ ms} - 95\text{ ms}$
Executes on every 3rd frame (Smart Frame Skipping) to maintain low CPU thermal load while keeping detection responsive.

8. Teammate Demo Pre-Flight Checklist

Pre-Flight Verification Items

- [ ] Laptop connected to local router; IP address updated in `index.html` (`LAPTOP_IP`).
- [ ] Python dependencies verified: `opencv-python`, `websockets`, `networkx`, `requests`.
- [ ] All 3 terminals running active processes without traceback errors.
- [ ] Marker ID 0 (Node A) and Marker ID 3 (Node D) placed at start and end testing points.
- [ ] Smartphone connected to HTTPS tunnel link; audio volume turned up.